

B.C.A. (Sem. II) Examination, 2026

(Session 2025–28)

2 BCA 1**Digital Computer Organization**

Paper Code : 211601

Time : 3:00 Hours**Full Marks : 100****Pass Marks : 40**

Note: Candidates are required to give answers in their own words as far as practicable. The questions are of equal value. Answer any five questions.

EXAMINATION PAPER WITH COMPREHENSIVE MODEL SOLUTIONS**Question 1**

Convert the following numbers into equivalent binary numbers:

(a) $(45.25)_{10} = (\quad)_2$

(b) $(73)_8 = (\quad)_2$

(c) $(156)_{10} = (\quad)_2$

(d) $(111010.101)_2 = (\quad)_8$

Solution 1

(a) Convert $(45.25)_{10}$ to Binary:

- *Integer Part* $(45)_{10}$:

$$45 \div 2 = 22 \quad \text{Remainder 1 (LSB)}$$

$$22 \div 2 = 11 \quad \text{Remainder 0}$$

$$11 \div 2 = 5 \quad \text{Remainder 1}$$

$$5 \div 2 = 2 \quad \text{Remainder 1}$$

$$2 \div 2 = 1 \quad \text{Remainder 0}$$

$$1 \div 2 = 0 \quad \text{Remainder 1 (MSB)}$$

Reading remainders bottom to top: $(45)_{10} = (101101)_2$.

- *Fractional Part* $(0.25)_{10}$:

$$0.25 \times 2 = 0.50 \quad \text{Integer part: 0}$$

$$0.50 \times 2 = 1.00 \quad \text{Integer part: 1}$$

Reading integer parts top to bottom: $(0.25)_{10} = (0.01)_2$.

- **Final Result:** $(45.25)_{10} = (101101.01)_2$.

(b) Convert $(73)_8$ to Binary: Each octal digit maps directly to a 3-bit binary group:

$$7 \rightarrow 111_2, \quad 3 \rightarrow 011_2$$

Final Result: $(73)_8 = (111011)_2$.**(c) Convert $(156)_{10}$ to Binary:**

$$156 = 128 + 16 + 8 + 4 = 2^7 + 2^4 + 2^3 + 2^2 = (10011100)_2$$

Final Result: $(156)_{10} = (10011100)_2$.**(d) Convert $(111010.101)_2$ to Octal:** Group bits into triplets of 3 starting from the binary point:

$$\text{Integer part: } \underbrace{111}_7 \underbrace{010}_2 \rightarrow (72)_8$$

$$\text{Fractional part: } .\underbrace{101}_5 \rightarrow (.5)_8$$

Final Result: $(111010.101)_2 = (72.5)_8$.

Question 2

Perform the following additions using 8-bit 2's complement arithmetic. Also state whether overflow occurs:

- (a) +38 and +47
- (b) -28 and +16
- (c) +72 and +61
- (d) -55 and -30

Solution 2

In 8-bit 2's complement arithmetic, representable range is $[-128, +127]$. Overflow occurs when adding two numbers of the same sign produces a result of the opposite sign (i.e., Carry-in to MSB $C_7 \neq$ Carry-out C_8 , so $C_7 \oplus C_8 = 1$).

(a) +38 and +47:

$$\begin{aligned} +38 &= 00100110_2 \\ +47 &= 00101111_2 \\ \text{Sum} &= \mathbf{01010101_2} \quad (+85_{10}) \end{aligned}$$

Carries: $C_7 = 0, C_8 = 0 \implies C_7 \oplus C_8 = 0$.

Result: $01010101_2 = +85$. **No Overflow** ($+85 \in [-128, +127]$).

(b) -28 and +16:

$$\begin{aligned} +28 &= 00011100_2 \implies -28 = 2\text{'s comp} = 11100100_2 \\ +16 &= 00010000_2 \\ \text{Sum} &= \mathbf{11110100_2} \quad (-12_{10}) \end{aligned}$$

Carries: $C_7 = 0, C_8 = 0 \implies C_7 \oplus C_8 = 0$.

Result: $11110100_2 = -12$. **No Overflow**.

(c) +72 and +61:

$$\begin{aligned} +72 &= 00100100_2 \\ +61 &= 00111101_2 \\ \text{Sum} &= \mathbf{10000101_2} \quad (\text{Sign bit} = 1, \text{negative!}) \end{aligned}$$

Carries: $C_7 = 1, C_8 = 0 \implies C_7 \oplus C_8 = 1$. Expected sum is $+133 > +127$.

Result: Overflow Occurs! The sum exceeds $+127$.

(d) -55 and -30:

$$\begin{aligned} +55 &= 00110111_2 \implies -55 = 11001001_2 \\ +30 &= 00011110_2 \implies -30 = 11100010_2 \\ \text{Sum} &= (1) \mathbf{10101011_2} \quad (-85_{10}) \end{aligned}$$

Carries: $C_7 = 1, C_8 = 1 \implies C_7 \oplus C_8 = 0$.

Result: $10101011_2 = -85$. **No Overflow** (Discard end carry $C_8 = 1$).

Question 3

What is a logic gate? Explain AND, OR and NOT gates with their truth tables. Also describe the concept of a universal gate.

Solution 3

Logic Gate Definition: An electronic circuit that operates on one or more binary inputs to produce a single binary output based on logical rules.

1. **AND Gate:** Output $Y = A \cdot B$ is HIGH (1) if and only if all inputs are HIGH (1).

A	B	$Y = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

2. **OR Gate:** Output $Y = A + B$ is HIGH (1) if at least one input is HIGH (1).

A	B	$Y = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

3. **NOT Gate (Inverter):** Output $Y = \bar{A}$ is the binary complement of input A .

A	$Y = \bar{A}$
0	1
1	0

Concept of Universal Gates: A **Universal Gate** is a gate that can implement any Boolean function or basic logic operation (AND, OR, NOT) without requiring any other type of gate.

- **NAND and NOR** gates are universal gates.
- Example using NAND:
 - **NOT:** Connect both inputs of NAND together: $Y = \overline{A \cdot A} = \bar{A}$.
 - **AND:** Pass NAND output through a NAND inverter: $Y = \overline{\overline{A \cdot B}} = A \cdot B$.
 - **OR:** Invert inputs using NAND before NANDing: $Y = \overline{\bar{A} \cdot \bar{B}} = A + B$.

Question 4

Define Boolean Algebra. State any five basic laws of Boolean algebra and verify any two laws with suitable examples.

Solution 4

Definition of Boolean Algebra: Boolean Algebra is a mathematical system operating on a set of binary values $B = \{0, 1\}$ using logical operators AND ($\cdot$), OR ($+$), and NOT ($\bar{}$) to analyze and simplify digital logic circuits.

Five Basic Laws of Boolean Algebra:

1. **Commutative Law:** $A + B = B + A$ and $A \cdot B = B \cdot A$
2. **Associative Law:** $A + (B + C) = (A + B) + C$ and $A \cdot (B \cdot C) = (A \cdot B) \cdot C$
3. **Distributive Law:** $A \cdot (B + C) = A \cdot B + A \cdot C$ and $A + (B \cdot C) = (A + B) \cdot (A + C)$
4. **Identity Law:** $A + 0 = A$ and $A \cdot 1 = A$
5. **Complement Law:** $A + \bar{A} = 1$ and $A \cdot \bar{A} = 0$

Verification of Two Laws:

Verification 1: Distributive Law $A + (B \cdot C) = (A + B) \cdot (A + C)$

A	B	C	B·C	A + B·C	(A+B)	(A+C)	(A+B)·(A+C)
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

Since the columns for $A + (B \cdot C)$ and $(A + B) \cdot (A + C)$ are identical for all combinations, the law is verified.

Verification 2: De Morgan's Law $\overline{A + B} = \bar{A} \cdot \bar{B}$

A	B	A+B	$\overline{A+B}$	$\bar{A}$	$\bar{B}$	$\bar{A} \cdot \bar{B}$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Columns 4 and 7 match perfectly, verifying $\overline{A + B} = \bar{A} \cdot \bar{B}$.

Question 5

Explain any two of the following: (a) Principle of duality (b) De Morgan's laws (c) Karnaugh Map (d) BCD

Solution 5

(a) Principle of Duality: The Principle of Duality states that any true Boolean algebraic identity remains valid if:

1. Every AND ($\cdot$) operator is replaced with an OR ($+$) operator, and vice-versa.
2. Every binary constant 0 is replaced with 1, and vice-versa.

Example: Dual of identity $A + 0 = A$ is $A \cdot 1 = A$. Dual of $A \cdot (B + C) = A \cdot B + A \cdot C$ is $A + (B \cdot C) = (A + B) \cdot (A + C)$.

(b) De Morgan's Laws: De Morgan's laws provide mathematical rules for finding the complement of logical expressions:

- **First Law:** The complement of a sum equals the product of individual complements: $\overline{A + B} = \bar{A} \cdot \bar{B}$.
- **Second Law:** The complement of a product equals the sum of individual complements: $\overline{A \cdot B} = \bar{A} + \bar{B}$.

(c) Karnaugh Map (K-Map): A Karnaugh Map is a visual tabular method used to minimize Boolean expressions systematically without complex algebraic manipulations. Grid cells are arranged in ****Gray Code order**** (neighboring cells differ by only 1 bit), allowing adjacent minterms to be grouped into blocks of 2, 4, 8, or 16 to eliminate redundant variables.

(d) Binary Coded Decimal (BCD): BCD is a class of binary encodings where each decimal digit (0 through 9) is independently represented by its 4-bit binary equivalent (0000_2 to 1001_2). Codes 1010_2 to 1111_2 are invalid in 8421 BCD.

Example: $(47)_{10} = \underbrace{0100}_4 \underbrace{0111}_7 = (01000111)_{\text{BCD}}$.

Question 6

Draw and explain a Half-Adder circuit. Show how a Half-Adder can be constructed using basic logic gates and give its truth table.

Solution 6

Half-Adder Overview: A Half-Adder is a combinational logic circuit that adds two single binary digits A and B , generating a **Sum (S)** and a **Carry (C)** output.

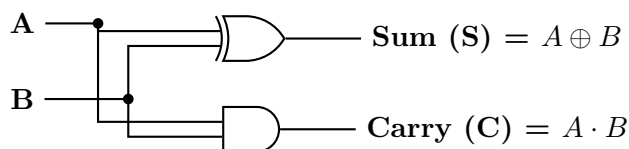
Truth Table for Half-Adder:

Inputs		Outputs	
A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions:

- Sum (S) = $\bar{A}B + A\bar{B} = A \oplus B$ (XOR operation)
- Carry (C) = $A \cdot B$ (AND operation)

Logic Diagram using Basic & XOR Gates:



Construction using Basic Gates (AND, OR, NOT) only:

$$S = A\bar{B} + \bar{A}B = \text{OR of two AND gates with inverted inputs}, \quad C = A \cdot B$$

Question 7

(a) Simplify the following Boolean expressions:

(i) $A(B + C') + A'BC$

(ii) $(A + B)(A' + C)(B + C)$

(b) Find the complement of the following expressions:

(i) $(A + B')C + D'$

(ii) $(X' + Y + Z)(X + Y')(Y + Z')$

Solution 7**(a) Simplification of Boolean Expressions:****(i) Simplify $F = A(B + C') + A'BC$:**

$$\begin{aligned}
F &= AB + AC' + A'BC \quad (\text{Expand brackets}) \\
&= AB + A'BC + AC' \\
&= B(A + A'C) + AC' \\
&= B(A + C) + AC' \quad (\text{Since } A + A'C = A + C) \\
&= \mathbf{AB + BC + AC'}
\end{aligned}$$

(ii) Simplify $G = (A + B)(A' + C)(B + C)$:

$$\begin{aligned}
(A + B)(A' + C) &= AA' + AC + A'B + BC = 0 + AC + A'B + BC \\
&= AC + A'B + BC \quad (\text{By Consensus Theorem, } BC \text{ is redundant}) \\
&= AC + A'B
\end{aligned}$$

Now multiply by $(B + C)$:

$$\begin{aligned}
G &= (AC + A'B)(B + C) = ABC + ACC + A'BB + A'BC \\
&= ABC + AC + A'B + A'BC = AC(1 + B) + A'B(1 + C) \\
&= \mathbf{AC + A'B}
\end{aligned}$$

(b) Finding Complements using De Morgan's Laws:**(i) Complement of $F = (A + B')C + D'$:**

$$\begin{aligned}
F' &= [((A + B')C) + D']' \\
&= [(A + B')C]' \cdot (D')'' \quad (\text{De Morgan's First Law}) \\
&= [(A + B')' + C'] \cdot D \quad (\text{De Morgan's Second Law}) \\
&= [(A' \cdot B'') + C'] \cdot D \\
&= (\mathbf{A'B + C'})\mathbf{D} = \mathbf{A'BD + C'D}
\end{aligned}$$

(ii) Complement of $G = (X' + Y + Z)(X + Y')(Y + Z')$:

$$\begin{aligned}
G' &= [(X' + Y + Z)(X + Y')(Y + Z')]'' \\
&= (X' + Y + Z)' + (X + Y')' + (Y + Z')' \\
&= (\mathbf{XY'Z'}) + (\mathbf{X'Y}) + (\mathbf{Y'Z})
\end{aligned}$$

Question 8

Explain Karnaugh Map (K-Map) in detail. Describe the procedure for simplifying Boolean expressions using a K-Map. Simplify the following Boolean function using a 4-variable K-Map and draw the corresponding logic circuit:

$$F(A, B, C, D) = \sum(0, 2, 3, 6, 7, 8, 10, 11, 14, 15)$$

Solution 8**Procedure for K-Map Simplification:**

1. Construct a 4×4 cell matrix representing 16 minterms m_0 to m_{15} .
2. Label rows (AB) and columns (CD) using Gray Code (00, 01, 11, 10).
3. Enter '1' in cells corresponding to specified minterms in $\sum(\dots)$.
4. Form maximal groups of 1s in powers of 2 ($16 \rightarrow 8 \rightarrow 4 \rightarrow 2$).
5. Sum the simplified product term of each group to get the minimal Sum-of-Products (SOP) expression.

K-Map for $F(A, B, C, D) = \sum(0, 2, 3, 6, 7, 8, 10, 11, 14, 15)$:

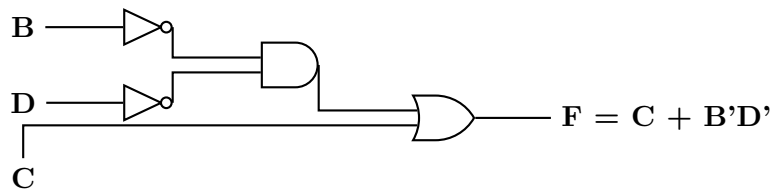
$AB \setminus CD$	00 ($C'D'$)	01 ($C'D$)	11 (CD)	10 (CD')
00 ($A'B'$)	1 (m_0)	0 (m_1)	1 (m_3)	1 (m_2)
01 ($A'B$)	0 (m_4)	0 (m_5)	1 (m_7)	1 (m_6)
11 (AB)	0 (m_{12})	0 (m_{13})	1 (m_{15})	1 (m_{14})
10 (AB')	1 (m_8)	0 (m_9)	1 (m_{11})	1 (m_{10})

Group Formations:

- **Group 1 (Octet):** All 8 cells in columns 11 and 10 ($m_2, m_3, m_6, m_7, m_{10}, m_{11}, m_{14}, m_{15}$). Variables A, B, D change state; C remains 1. $\Rightarrow C$.
- **Group 2 (Corner Quad):** 4 corners (m_0, m_2, m_8, m_{10}). Across top/bottom rows $AB = 00, 10 \Rightarrow B'$. Across left/right columns $CD = 00, 10 \Rightarrow D'$. $\Rightarrow B'D'$.

Minimal Boolean Function:

$$F(A, B, C, D) = C + B'D'$$

Logic Circuit Diagram:

Question 9

What is a Flip-Flop? Explain the S-R Flip-Flop with a suitable logic diagram and Truth table. Mention its applications.

Solution 9

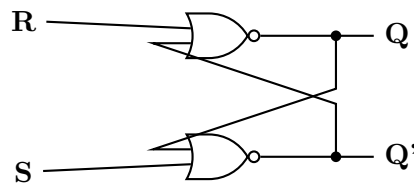
Definition of Flip-Flop: A Flip-Flop is a 1-bit bistable multivibrator circuit capable of serving as a memory element. It retains its binary output state (0 or 1) indefinitely until modified by an external input or clock pulse.

S-R (Set-Reset) Flip-Flop Operation: The SR Flip-Flop has two control inputs: **S (Set)** and **R (Reset)**, and two complementary outputs: Q and Q' .

Truth / Characteristic Table:

S	R	Q_{n+1}	Q'_{n+1}	Action / State
0	0	Q_n	Q'_n	No Change (Hold)
0	1	0	1	Reset
1	0	1	0	Set
1	1	Indeterminate	Indeterminate	Invalid / Forbidden

Logic Circuit Diagram (Using Cross-Coupled NOR Gates):



Applications of Flip-Flops:

1. Storage registers and SRAM memory units.
2. Frequency division and digital counters (Ripple / Synchronous).
3. Switch debouncing circuits.

Question 10

Write short notes on any two of the following: (i) Registers (ii) Multiplexers (iii) Sequential circuits (iv) ROM and RAM

Solution 10

(i) Registers: A Register is a high-speed digital storage device consisting of a group of flip-flops connected in cascade, sharing a common clock signal. An n -bit register stores n bits of binary information.

- **Shift Registers:** Move data serially or in parallel (SISO, SIPO, PISO, PIPO).
- **CPU Registers:** Accumulator (AC), Program Counter (PC), Instruction Register (IR).

(ii) Multiplexers (MUX): A Multiplexer (Data Selector) is a combinational circuit that selects one of 2^n input signals and routes it to a single output line based on the state of n selection lines.

Inputs: 2^n , Select Lines: n , Output: 1

Applications: Data routing, parallel-to-serial conversion, Boolean function generation.

(iii) Sequential Circuits: Unlike combinational circuits whose outputs depend solely on present inputs, a **Sequential Circuit** produces outputs based on both current inputs and the **past history** (stored memory state).

Sequential Block Structure: Present Inputs $\rightarrow$ Combinational Logic $\leftrightarrow$ Memory Elements (Flip-Flops) $\rightarrow$ Outputs.

(iv) ROM and RAM:

Feature	RAM (Random Access Memory)	ROM (Read Only Memory)
Volatility	Volatile (Data lost when powered off)	Non-volatile (Data permanently retained)
Operation	Read and Write capability	Read-only capability
Primary Use	Main memory holding active CPU programs	Storing BIOS / firmware bootloader
Types	SRAM, DRAM	PROM, EPROM, EEPROM